Introduction

Infrastructure & DevOps

Aurélien Leygues (aurelien.leygues@gmail.com)

Objectifs

Connaissances à acquérir dans le cadre de ce module :

- Rappels et mise en oeuvre des concepts avancés de Docker
- Orchestrer de manière efficace des *containers* sur une architecture distribuée
- Combiner CI & CD pour garantir la qualité et la maintenabilité d'une application

Planning

- Introduction, présentation de l'app support et Dockerization
- Mise en place d'une architecture distribuée P1 : mise en place de Docker Swarm et Ansible
- Mise en place d'une architecture distribuée P2 : ajout de Traefik et Portainer
- Ajouter le déploiement continue (*CD*) avec Watchtower
- Monitorer un *cluster* Docker Swarm
- Documentation complète du déploiement de la solution

Mode d'évaluation

Dans le cadre du module, vous allez devoir *containeriser*, orchestrer et garantir la qualité et la maintenabilité d'une app, vous serez évalué·es sur :

- QCM individuel lors de la dernière séance
- la documentation complète de votre projet, à remettre le dimanche qui succède la dernière séance, le détail des attentes vous sera donné à l'avant dernière séance

Les 2 évaluations ont le même coefficient

Organisation générale

- Les contenus présentés seront disponibles sur MyGes
- Pour les questions, écrivez-moi plutôt à aurelien.leygues@gmail.com
- Les premières séances sont en travail individuel, les dernières seront par groupe

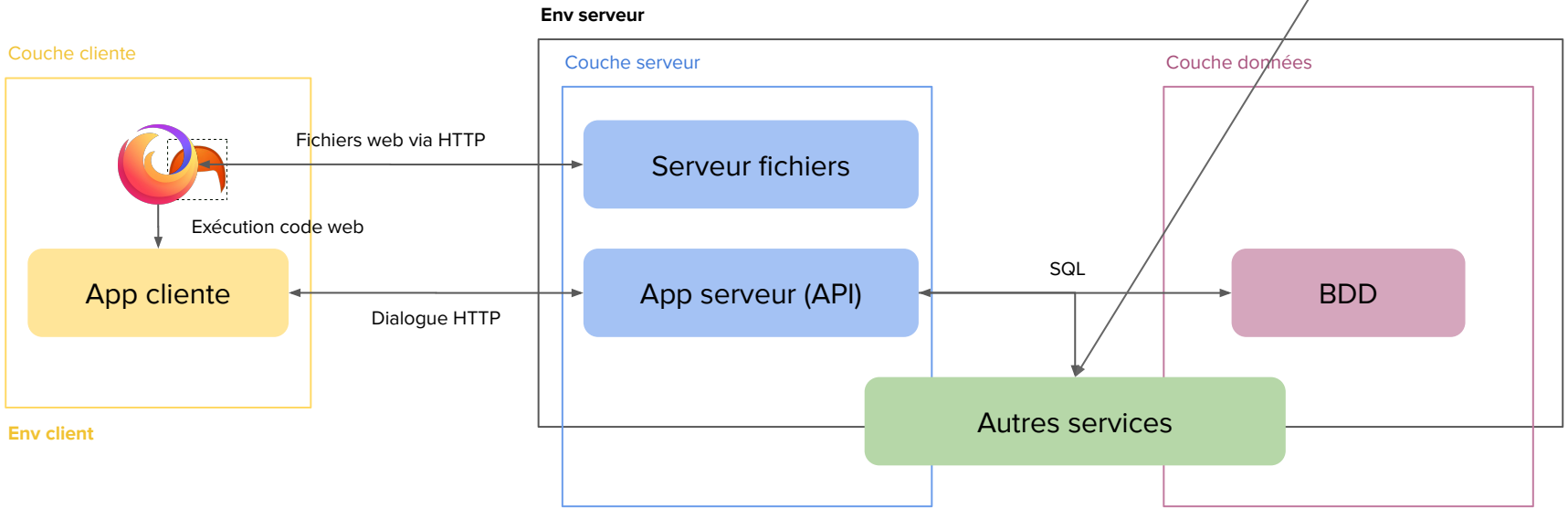
Rappel Docker

Un outil de containerisation incontournable

Architecture app (exemple)

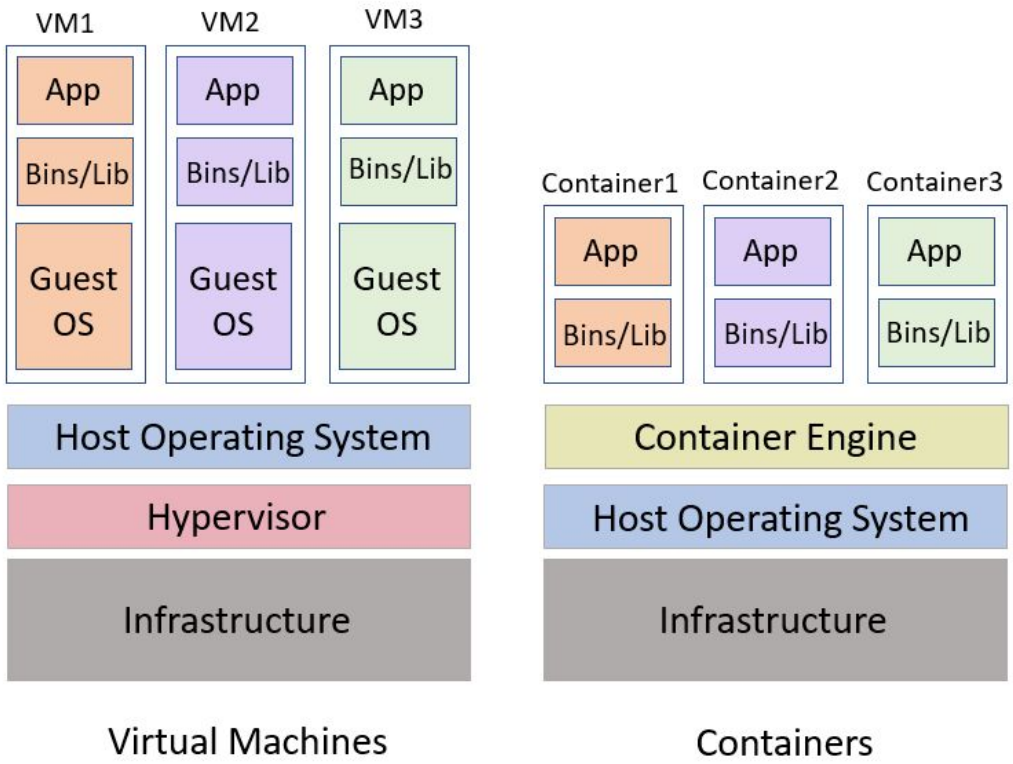
Selon les services, peuvent être côté couche données ou serveur, externes ou dans l'env serveur

Souvent, une solution (= application) regroupe une app cliente et une app serveur, qui communiquent avec d'autres logiciels (BDD, services, etc)



⇒ attention aux mots "serveur" et "application" qui peuvent avoir **beaucoup** de sens différents !

Containerization VS virtualization



- Les VM réservent des ressources, les containers les partagent dans des limites configurées
- Les containers partagent des couches logiciels (OS et plus !), pas les VM
- Les VM offrent une isolation parfaite, les containers quasi-parfaite

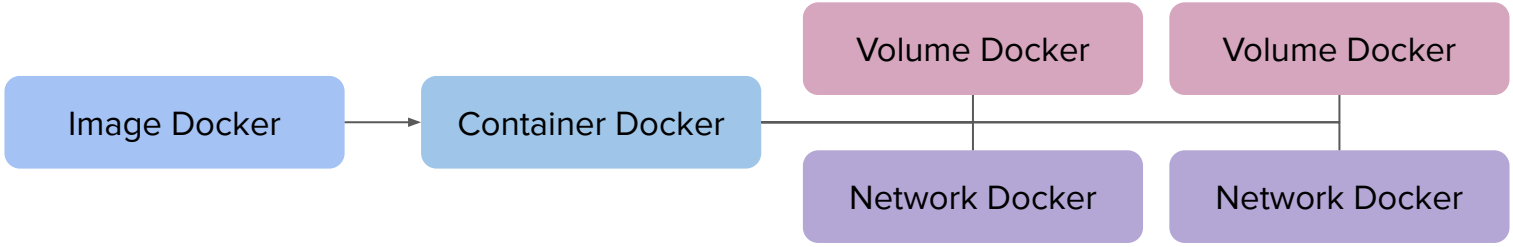
Docker

Nos applications sont exécutées dans un environnement défini, maîtrisé et stable dans le temps :

- Indépendance des OS, parfait pour dev et prod
- Aucun dysfonctionnement lié aux MAJ système
- Maintenabilité renforcée

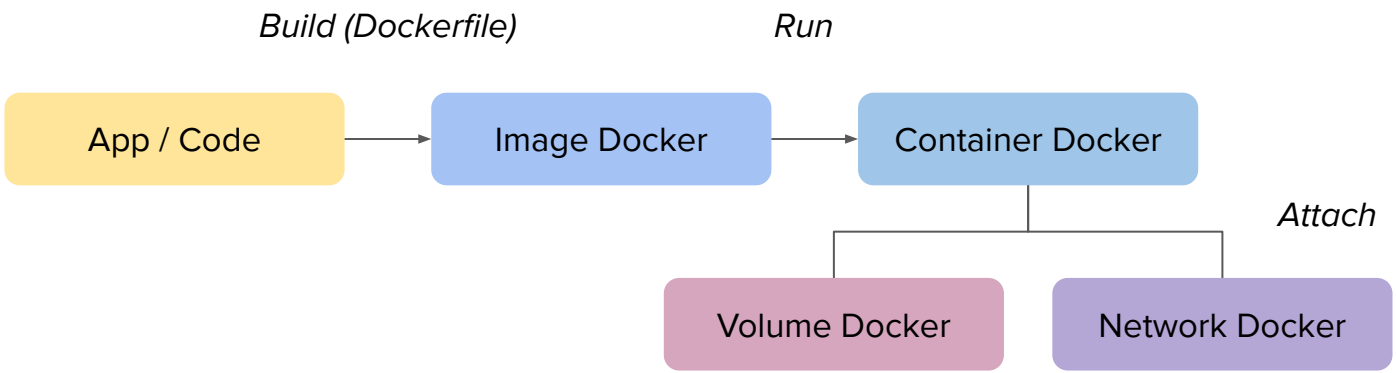
Terminologie Docker

- **Images** : version d'une app à un instant T, dans un environnement figé, c'est un exécutable
- **Containers** : exécution d'une image
- **Volume** : unité de stockage montée sur un container, pour péreniser les données
- **Réseaux** : communication inter containers



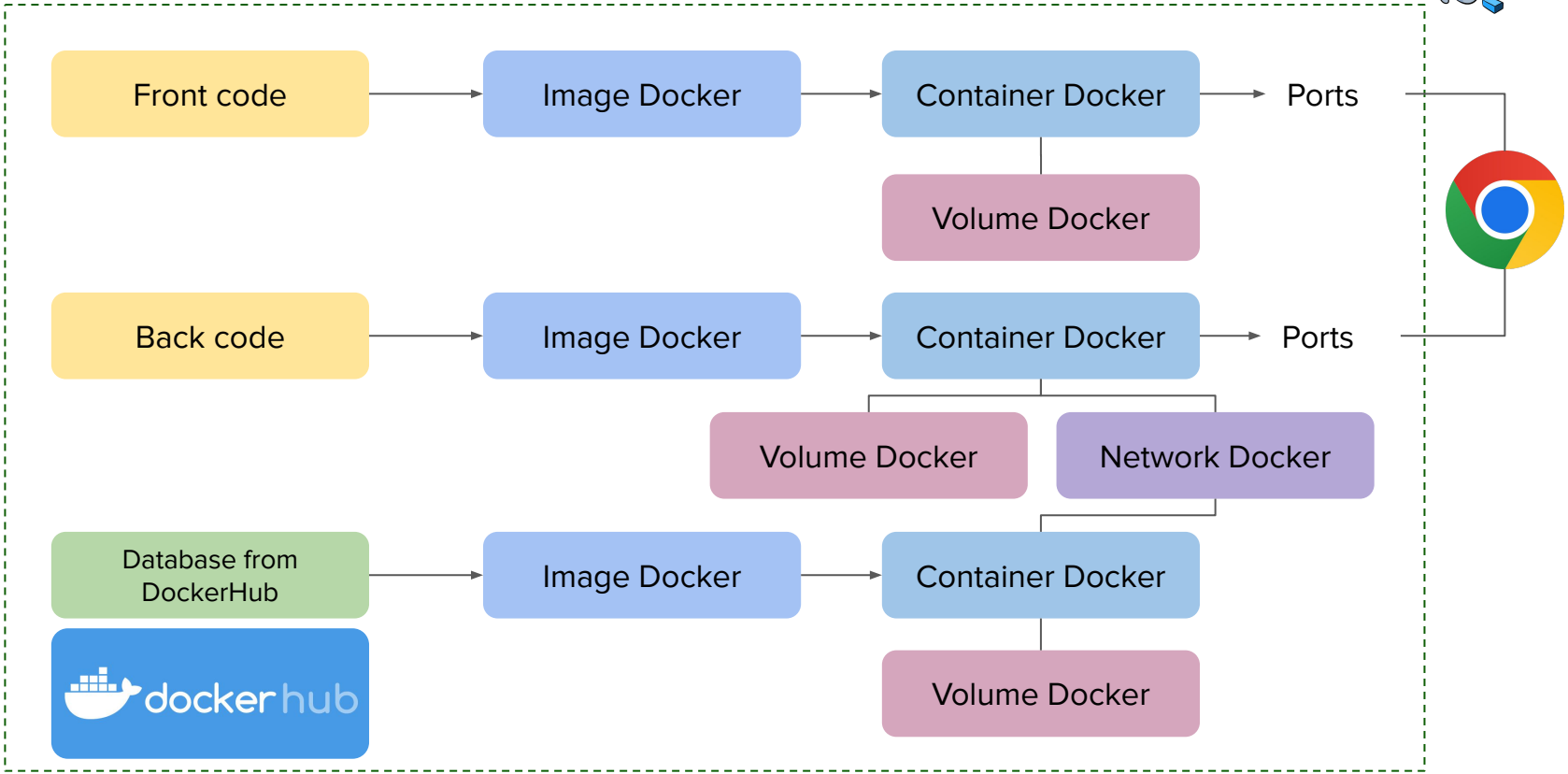
Dockerizer une app

Une app = un programme à exécuter, donc soit le *front*, soit le *back*



Orchestrer Docker

Docker compose (*compose.yml*)



Zoom sur le *build*

- La construction d’une image est décrite dans un Dockerfile
- Organisation en couches mises en cache et ré-utilisées
- Héritage d’images pour ne pas réinventer la roue

Héritage

Couches

```
# Dockerfile
FROM node:ubuntu

WORKDIR /app

COPY package.json package.json
RUN npm i

COPY src src
COPY tsconfig.json tsconfig.json

CMD npm start
```

```
# Dockerfile
FROM node:ubuntu

WORKDIR /app

COPY package.json package.json
RUN npm i

COPY src src
COPY tsconfig.json tsconfig.json

CMD npm start
```

Build → Image V1

Modification de **src/index.ts**

```
# Compilation
(CACHE) FROM node:ubuntu

(CACHE) WORKDIR /app

(CACHE) COPY package.json…
(CACHE) RUN npm i

COPY src src
COPY tsconfig.json tsconfig.json

CMD npm start
```

Image V2 ← *Build*

Zoom sur le *build*

- À chaque changement, on reconstruit (*live reloading* possible avec les volumes)
- Le cache permet de compiler très rapidement à condition de bien s'organiser
- **`package.json`** et **`npm i`** doivent être les premières couches
- Enfin on exécute l'image ou on la pousse (DockerHub par exemple)

Docker nous permet de développer et exécuter des apps dans des environnements connus, définis et stables dans le temps. Docker permet entre autres de faciliter le déploiement, les tests, et bien plus encore !

⇒ Présentation de l'app

My Favorite Places

Exercice 1 - Dockerization du projet :

1. Prenez connaissance du projet, testez le serveur (dossier server) avec **`yarn dev`**, faites une requête à l'API avec Bruno
2. Créez un Dockerfile pour dockerizer le serveur, testez de nouveau avec Bruno
3. Créez un **`compose.yml`** à la racine du projet pour lancer le serveur et une BDD PgSQL
4. Testez le client React puis dockerizez-le, et ajoutez-le au **`compose.yml`**, vérifiez le tout
5. Réflechissez : comment communique la BDD et le serveur ? Et le serveur et le *front* ?

Pour aller plus loin :

1. Dans **`compose.yml`**, faites le nécessaire pour que le serveur ne démarre qu'une fois la BDD prête
2. Faite en sorte que tous les services redémarrent automatiquement, sauf s'ils sont stoppés manuellement
3. Documentez-vous sur les *Docker build stages* et optimisez vos images Docker (l'image du serveur doit perdre ~100 Mo pour une version de prod)

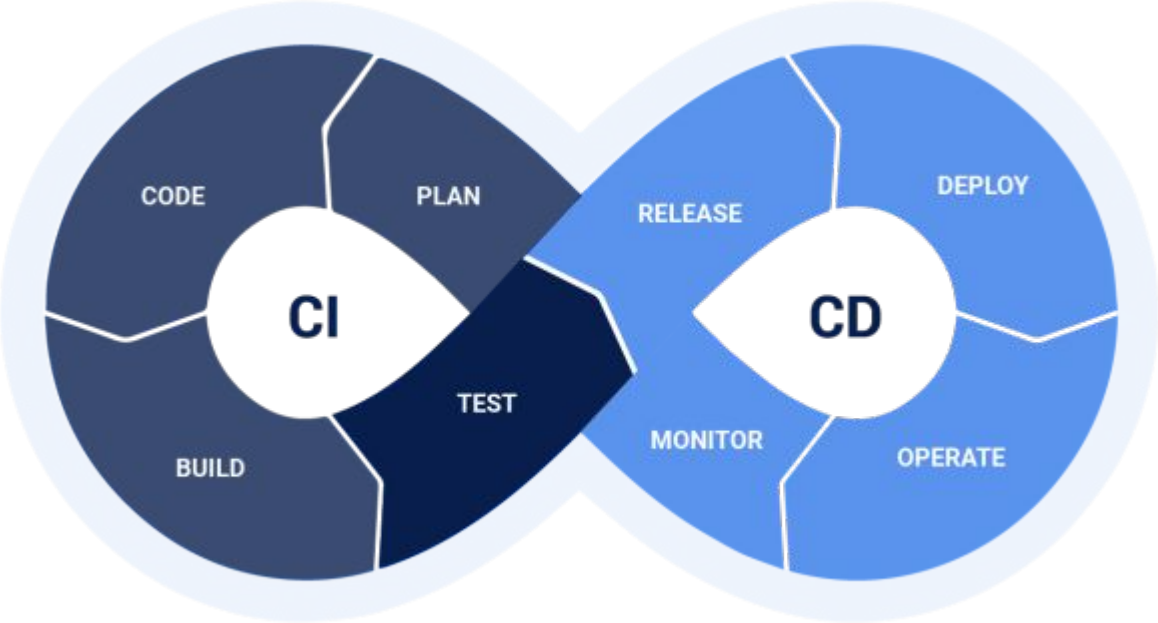
Intégration continue

Assurer automatiquement la qualité du projet

Principe

L'intégration continue (CI) s'inscrit dans les thématique DevOps et a pour objectif d'exécuter automatiquement des tâches de compilation et de test en réaction à des changements dans le code source publié (poussé sur Git)

CI & CD ⇒ approche DevOps



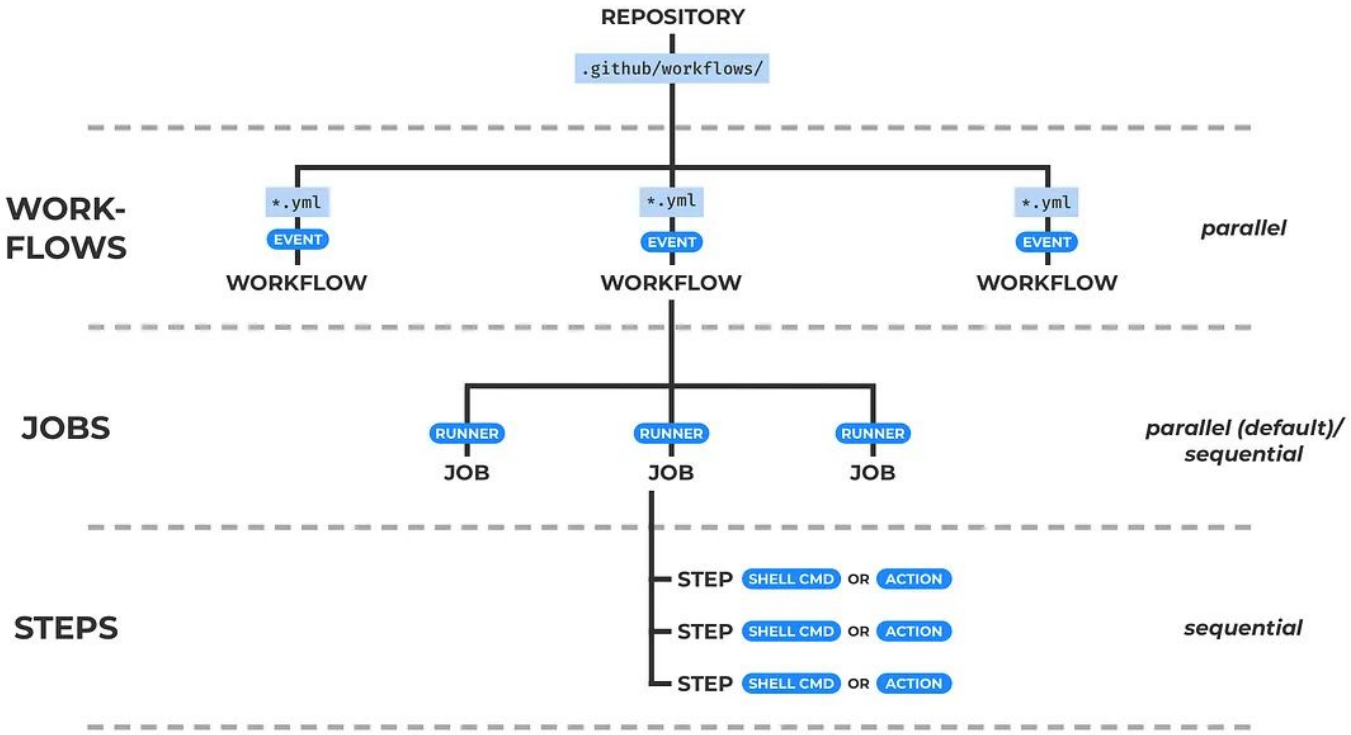
Github Actions

La CI doit être exécutée par des serveurs, Github met à disposition des serveurs pour nos CI :

- Gratuits pour les projets publics et payant (avec quelques centaines de minutes gratuites) pour les privés
- Basé sur le principe d'actions réutilisables qui s'exécutent en fonction de règles de déclenchement
- Se configure avec des fichiers YAML en respectant l'arborescences et les paramètres exigés par Github

Gitlab propose l'équivalent !

Github Actions



Exercice 2 - Mettre en place une CI :

1. Créez votre propre repo public pour MFP, et en vous appuyant sur la doc des Github Actions, mettez en place une CI qui compile automatiquement les images de *front* et *back* à chaque *push* sur **`main`** ou **`master`** (taguées en latest)

2. Ajoutez au moins un test dans votre projet (par exemple, testez la fonction **`getDistance.ts`** avec Jest), puis ajoutez un script NPM pour exécuter le test, enfin mettez à jour votre CI pour qu'elle exécute les tests à chaque *push* sur **`main`** ou **`master`**

3. Mettez à jour votre repo pour interdire les push directement sur **`main`** ou **`master`**, et conditionnez la possibilité de merge une PR au fait que les tests réussissent, puis testez (nouvelle branche → casser volontairement la fontion **`getDistance.ts`** → pousser et créer une PR → elle doit être impossible à merger)

Exercice 3 - Vérifiez les images produites :

Après la question 2.2, vous devriez avoir des images Docker *front* et *back* disponibles sur votre repo Github, créez une copie de **`compose.yml`** appelée **`compose.prod.yml`** qui ne compile plus les images depuis votre code, mais s'appuie sur les version disponnibles sur Github de vos images (vos instructions **`build:`** doivent disparaître en faveur des **`image:`**)

Exécutez : tout doit correctement fonctionner pour faire la suite

Pour aller plus loin :

1. Modifiez votre CI pour tester et compiler le *back* ou le *front* uniquement si des fichiers du dossier *back* ou du dossier *front* ont été modifiés

2. Ajouter un test d'API Bruno et executez-le en ligne de commande (documentation), vérifiez que le dossier de la collection Bruno est bien contenu dans votre dossier de projet (donc sur Github), puis modifiez votre CI pour que le test Bruno soit bien exécuté (indice : il faut d'abord compiler + exécuter votre API dans la CI pour exécuter le test Bruno !)

3. Ajoutez un test E2E Cypress (par exemple, se créer un compte) puis essayez de l'ajouter aux tests exécutés par votre CI (indice : il faut lancer le *front*, le *back* et la BDD, puis ensuite le test dans la CI)

À vous de jouer !